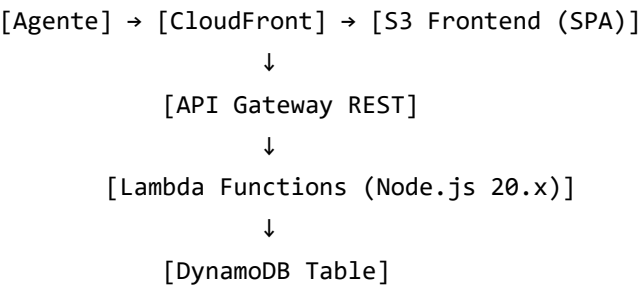


Caso Práctico – Ingeniero DevOps

Contexto

Una aseguradora colombiana cuenta con una aplicación serverless de **registro de siniestros vehiculares** desplegada en AWS. La aplicación está en producción y atiende a agentes que reportan incidentes desde terreno.

Arquitectura Actual



Componentes desplegados

Capa	Servicio	Detalle
CDN	CloudFront	Distribución con certificado ACM y WAF
Frontend	S3 Bucket	Aplicación SPA (React)
API	API Gateway	Endpoints REST
Compute	Lambda (arm64)	2 funciones de negocio
Data	DynamoDB	Tabla de siniestros (on-demand)
Seguridad	Secrets Manager + KMS	Credenciales de integraciones
IaC	CloudFormation (SAM)	Template parametrizado por ambiente

Funciones Lambda

Función	Responsabilidad
claims	Registro, consulta y actualización de siniestros
documents	Generación y almacenamiento de reportes PDF

Endpoints

Método	Path	Lambda
POST	/api/v1/claims	claims
GET	/api/v1/claims/{id}	claims
PATCH	/api/v1/claims/{id}	claims
POST	/api/v1/documents/report	documents

Ambientes

Ambiente	Propósito
dev	Desarrollo y pruebas unitarias
staging	QA, pruebas de integración
prod	Producción

Situación Actual (Problemas)

- Despliegue manual:** El equipo despliega ejecutando comandos de CloudFormation directamente desde sus máquinas locales o desde la consola AWS.
- Sin pipeline CI/CD:** No existe un flujo automatizado de build, test y deploy. Los cambios se suben directamente a las ramas sin validaciones previas.
- Sin análisis de seguridad:** No se ejecutan escaneos de código estático (SAST), análisis de dependencias (SCA) ni validación de IaC antes del despliegue.

4. **Sin observabilidad:** Los logs existen en CloudWatch pero no hay dashboards, alertas ni trazabilidad end-to-end. Cuando hay un incidente, el diagnóstico es manual.
 5. **Configuración inconsistente:** Los archivos de environment se editan manualmente y no hay validación de que los parámetros sean correctos antes de desplegar.
 6. **Sin estrategia de rollback:** Si un despliegue falla en producción, el rollback es manual y puede tomar más de 1 hora.
 7. **Datos sensibles:** La aplicación maneja información personal de asegurados (PII) y debe cumplir con regulación de la Superintendencia Financiera de Colombia (SFC).
-

Entregables Esperados

El candidato debe proponer e implementar:

1. Pipeline CI/CD

Diseñar un pipeline completo (Jenkins o GitHub Actions) que cubra:

- Checkout → Build → Test → Security Scan → Deploy
- Despliegue diferenciado por ambiente
- Aprobación manual para producción

2. IaC y gestión de ambientes

- Implementar la infraestructura solicitada
- ¿Cómo garantizar consistencia entre dev, staging y prod?
- ¿Cómo validar los templates antes de desplegar?
- ¿Cómo manejar secretos sin exponerlos en el repositorio?

3. Estrategia de despliegue

- Proponer estrategia (blue/green, canary, rolling) para las Lambdas
- Definir mecanismo de rollback automático
- ¿Cómo desplegar el frontend (S3 + CloudFront invalidation)?

4. Seguridad integrada

- ¿Qué herramientas integrar en el pipeline?

- ¿Cómo manejar vulnerabilidades detectadas (bloqueo vs advertencia)?
- ¿Cómo cumplir con regulación SFC para datos de asegurados?

5. Observabilidad

- Proponer stack de monitoreo para la aplicación
- Definir alertas críticas
- ¿Cómo medir métricas DORA?

6. Integración de AI

- Proponer al menos 2 casos de uso de AI aplicados al flujo DevOps de esta aplicación

Criterios de Evaluación

Criterio	Peso
Diseño del pipeline CI/CD	25%
Estrategia de IaC y ambientes	20%
Seguridad integrada (DevSecOps)	20%
Observabilidad y operación	15%
Integración de AI	10%
Claridad y viabilidad de la propuesta	10%

Restricciones

- La solución debe ser implementable en AWS
- Debe considerar costos (preferir servicios serverless/managed)
- El entorno es regulado (SFC) — considerar auditoría y trazabilidad

 Notas Importantes

- **Despliegue funcional:** Se espera ver el despliegue de los recursos funcionando en una cuenta de AWS. Los servicios utilizados en esta arquitectura cuentan con capa gratuita (Free Tier), por lo que no generarán costos.
 - **Pipeline en Jenkins:** Se requiere la definición del `Jenkinsfile` con las etapas del pipeline. Entregarlo funcional (ejecutándose en un Jenkins real) es **opcional**; lo importante es la definición y estructura del pipeline.
 - **Dudas o preguntas:** Si tiene alguna duda o pregunta sobre la prueba técnica, puede hacerla llegar por el mismo correo donde se entregó la prueba.
-